## Atividade Prática:

### Teste Funcional e Implementação do Programa "Identifier"

**Objetivo da Atividade:**

Aplicar o critério funcional de **Particionamento em Classes de Equivalência** na lista de casos de teste elaborada na última aula, implementar a solução de software e automatizar a execução dos testes utilizando tecnologias de livre escolha.

Para fins de comparação, mantenha a lista elaborada na última aula e elabore uma nova lista considerando o critério.

**Contexto e Especificação do Programa:**

Vocês foram encarregados de implementar e validar o programa **Identifier**. A especificação formal estabelece que:

> *O programa Identifier determina se um identificador é válido. Um identificador válido deve começar com uma letra e conter apenas letras ou dígitos. Além disso, deve ter no mínimo um caractere e no máximo seis caracteres de comprimento.*

**Exemplos de Comportamento Esperado:**

- Identifier string -> Output: Válido
- Identifier stringmuitogrande -> Output: Inválido

**Tarefas a serem desenvolvidas:**

**1. Implementação do Programa**

- Implemente o algoritmo do programa *Identifier* utilizando a linguagem de programação de sua preferência. O código deve ser modular o suficiente para receber entradas e retornar se o identificador é válido ou não.

**2. Projeto de Casos de Teste (O Relatório)**

- Elabore um relatório técnico detalhando o processo de derivação dos casos de teste de acordo com o Particionamento em Classes de Equivalência.
- O documento deve seguir obrigatoriamente estes passos metodológicos:
    - **Passo 1:** A partir da especificação do software, identificar as classes de equivalência (definir as classes válidas e inválidas para comprimento, caractere inicial e tipos de caracteres contidos).
    - **Passo 2:** Gerar os casos de teste selecionando um elemento de cada classe definida no passo anterior.
    - **Adicional:** Demonstre como a análise de valor limite (ex: tamanhos de 0, 1, 6 e 7 caracteres) complementa os resultados do seu particionamento em classes de equivalência.

**3. Implementação e Execução dos Testes**

- Utilize um framework de testes adequado à linguagem escolhida (ex: JUnit, PyTest, Jest).
- A implementação de cada caso de teste no código deve refletir a estrutura formal de testes, deixando claro:
    - **Setup:** Inicialização e configuração dos pré-requisitos para o teste.
    - **Invocation:** A execução do software com a entrada específica do caso de teste.
    - **Assessment (Asserts):** A avaliação comparando o resultado obtido com o resultado esperado.

**Entregáveis:**

1. **Código-Fonte:** Link para o repositório contendo a implementação do programa e o código dos testes automatizados.
2. **Relatório Técnico (PDF):** Documento detalhando o passo a passo da derivação das classes de equivalência (Tarefa 2) e um breve resumo dos resultados da execução dos testes na aplicação final.